Sandia National Laboratories

Exceptional service in the national interest

The Structural Simulation Toolkit (SST)

Oxford-Sandia Discussions

Content by: SST Team

Wednesday, July 30

SAND2025-09722PE

References

Websites

- Information on installing SST, and links to everything else you see here
  - http://www.sst-simulator.org/
- Documentation on SST's key interfaces, overview of all the elements, and more!
  - http://sst-simulator.org/sst-docs/
- Code
  - https://github.com/sstsimulator

Important Pages

- Configuration File Format: http://sst-simulator.org/sst-docs/docs/guides/configuration/pythonConfigGuide
- Getting Started: http://sst-simulator.org/sst-docs/docs/guides/start
- SST-Core Doxygen: http://sst-simulator.org/SSTDoxygen/15.0.0_docs/html/
- Building SST
  - Quick start: http://sst-simulator.org/SSTPages/SSTBuildAndInstall_15dot0dot0_SeriesQuickStart/
  - Detailed: http://sst-simulator.org/SSTPages/SSTBuildAndInstall_15dot0dot0_SeriesDetailedBuildInstructions/

Introduction to SST

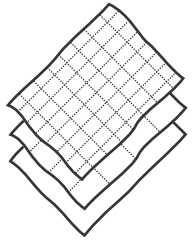
Simulation appears in many contexts

Research ***Prototyping*** ***Validation***

- When hardware or software doesn't exist yet
- When hardware exists but is difficult to access or measure

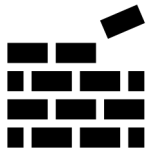
Reuse

- Different fidelities
- Different timescales
- Different system scales
- Related and sometimes intertwined problems



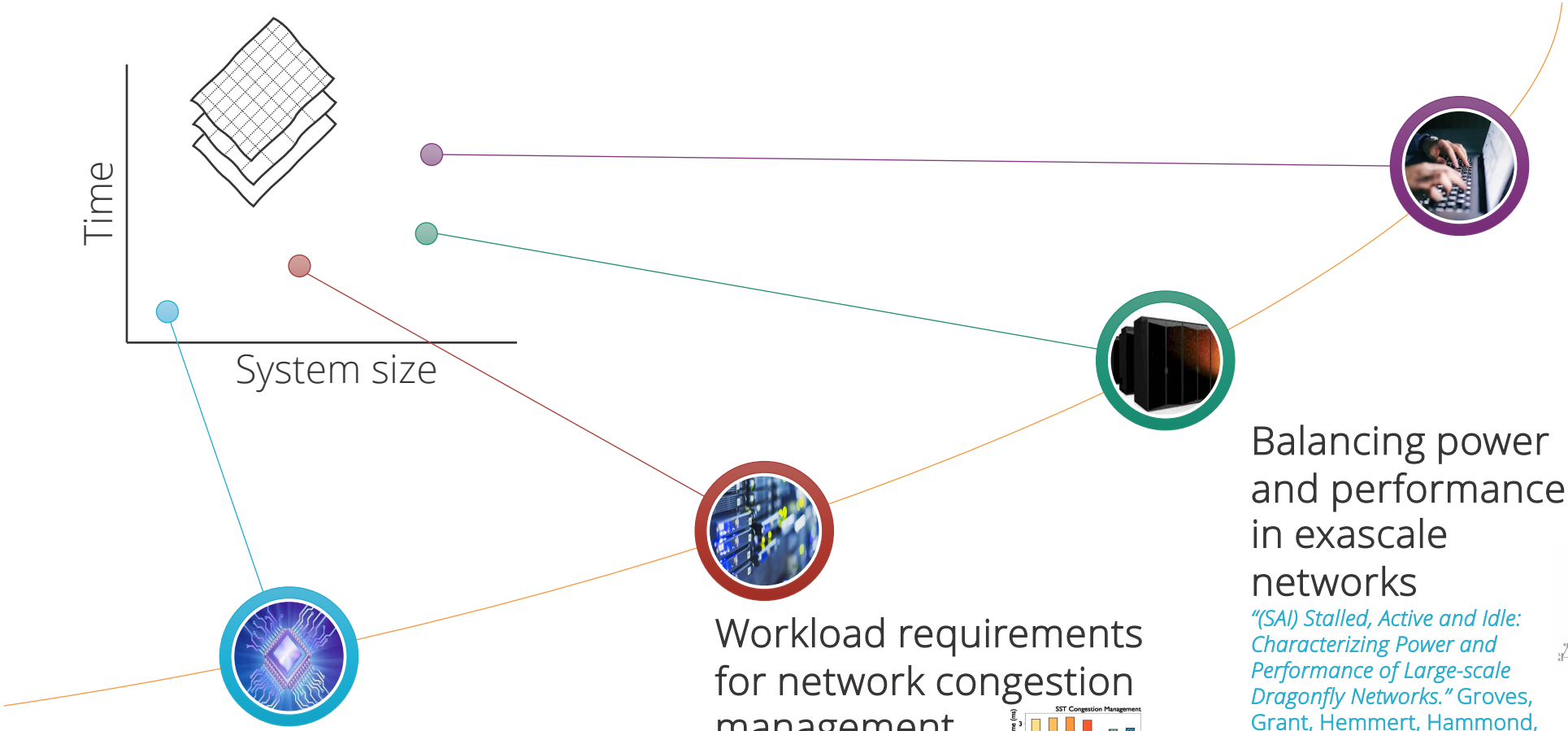
Multi-fidelity

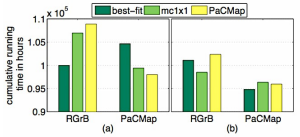
One ecosystem that can adapt to multiple needs and leverage community expertise



Modular

Simulation at many scales



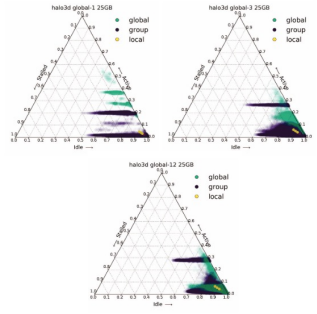


Job scheduling algorithms to improve runtime and throughput

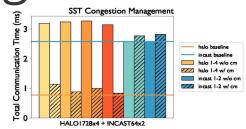
"PaCMap: Topology Mapping of Unstructured Communication Patterns onto Non-contiguous Allocations" Tuncer, Leung, Coskun. ICS, 2015.

Balancing power and performance in exascale networks

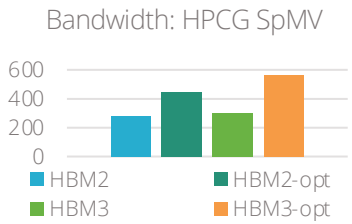
"(SAI) Stalled, Active and Idle: Characterizing Power and Performance of Large-scale Dragonfly Networks." Groves, Grant, Hemmert, Hammond, Levenhagen, Arnold. IEEE Cluster, 2016.



Workload requirements for network congestion management



"Exploration of Congestion Control Techniques on Dragonfly-class HPC Networks Through Simulation." McGlohon, Hemmert, Brown, Levenhagen, Chunduri, Ross, Carothers. PMBS Workshop. 2021.



On-chip network limits memory bandwidth

"HBM2/3 Evaluation on Many-Core CPU." Voskuilen, Gimenez, Peng, Moore, Gokhale. Report, Exascale Compute Project. 2018.

SST Overview

Why SST?

There is a rich selection of open-source simulators

But not a solid ecosystem for modeling systems

- Tightly-entangled components make modifications complex
  - E.g., assumptions about caching or address mapping are pervasive
- Most simulator integrations are ad-hoc, not lasting
- Significant performance problems with tying many simulators together

Wants:

- Enable “mix-and-match” of existing models to create custom systems
- Encourage disentangled models with clean interfaces for swapping functionality
  - Bricks not buildings
- Low effort, high performance parallel simulation
- Continuous path from low-fidelity/fast modeling to high-fidelity/slow models

The Structural Simulation Toolkit

Goals

- Create a standard architectural *simulation framework* for HPC*
- Ability to evaluate future systems on current and emerging workloads
- *Use supercomputers to design supercomputers*

Technical Approach

- **Parallel** Discrete Event core
  - With conservative optimization over MPI/Threads
- **Interoperability**
  - Node and system-scale models
- **Multi-scale**
  - Detailed (~cycle) and simple models that interoperate
- **Open**
  - Open Core, non-viral, modular

Status

- Parallel framework (*SST Core*)
- Integrated libraries of components (*Elements*)
- Current Release (15.0.0)
  - Two releases per year

Consortium

- "Best of Breed" simulation suite
- Used in industry, academia, and at labs

The SST Approach

Parallel Discrete-Event Simulator Framework (*SST Core*)

- Flexible framework enables multitude of custom “simulators”
- Demonstrated scaling to over 512 nodes running a million+ components

Comes with many built-in simulation models (*SST Elements*)

- Processors, memories, networks

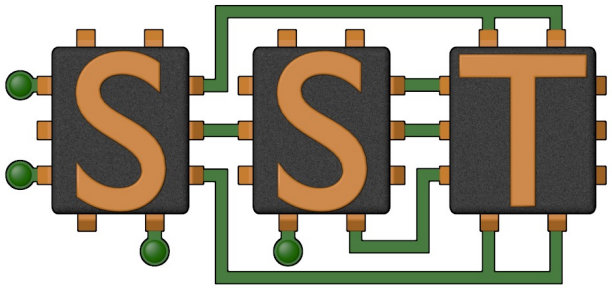
Open API

- Easily extensible with new models
- Modular framework
- Open-source core

Time-scale independent core

- Handles Micro-, Meso-, Macro-scale simulations

C++, Python



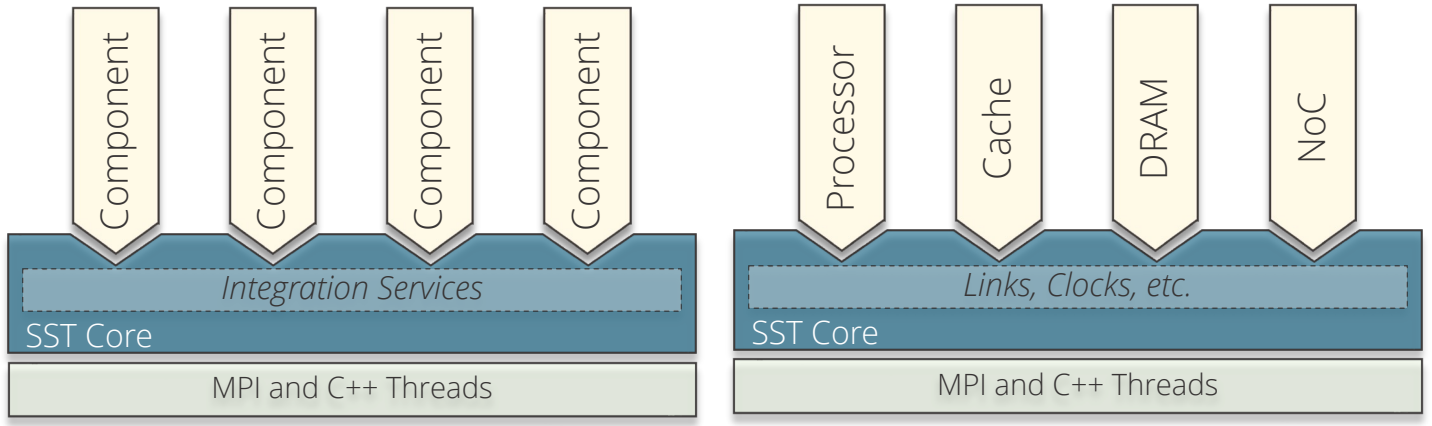
SST Architecture

SST **Core** framework

- The backbone of simulation
- Provides utilities and interfaces for simulation components (models)
  - Clocks, event exchange, statistics and parameter management, parallelism support, etc.

SST **Element** libraries

- Models that perform the actual simulation
- Elements include processors, memory, network, etc.
  - Leverages many existing simulators: Spike, DRAMSim3, Ramulator2, etc.



Building Blocks: 101

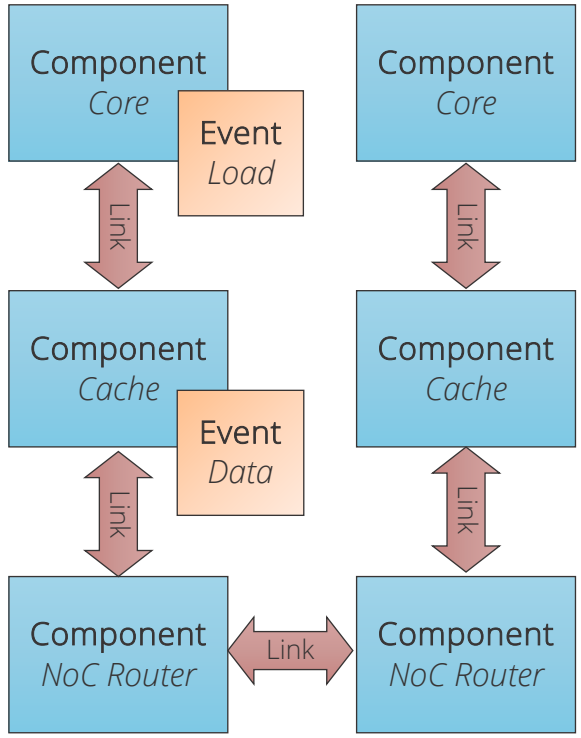
Systems are described as a set of **components**

Components are connected by **links**

Components communicate by sending **events** over links

When creating models for SST, developers map their models to this structure

When running models in SST, the user maps their desired system to this structure and tells SST which components to use and how they are linked



Element Library

A collection of components and other building blocks

Building Blocks: Restrictions

Developers:

- Components may **only** communicate with other components via events

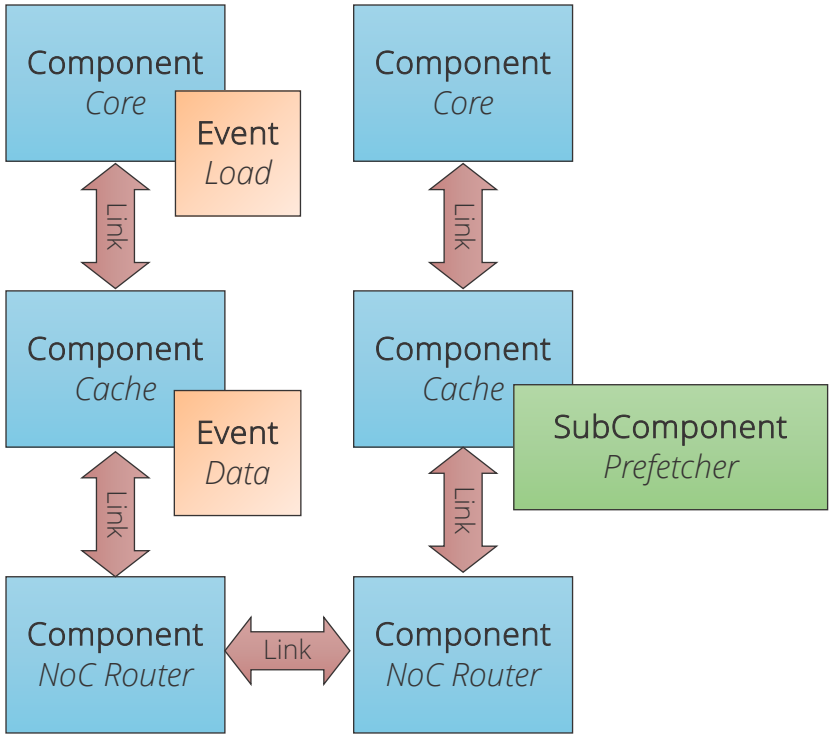
Users:

- Every event sent between components incurs a latency
- The latency cannot be zero and **the minimum latency is set by the user**

Building Blocks: 201

SubComponents and **Modules** allow users to swap in different functionality *within* a Component

- SubComponents are larger and might participate in event exchange
  - A prefetcher
- Modules are smaller, with very limited access to simulation APIs
  - A hash function



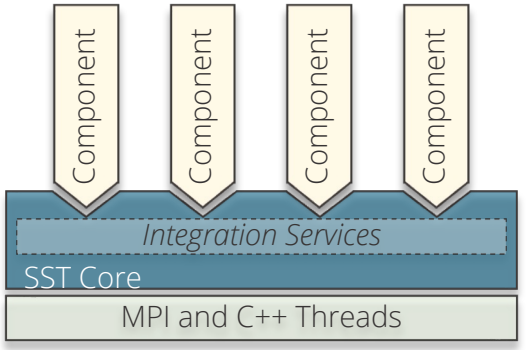
Standardized interfaces for certain types of components makes swapping components simpler

- Endpoint / Network: SimpleNetwork
- Processor / Memory: StandardMem

SST Code Structure

SST Core and **SST Elements** are compiled separately

- Element libraries *register* with the core
- External elements (not part of SST Elements) can also be registered with the core
  - Example at github.com/sstsimulator/sst-external-element
- Core maintains a database of registered libraries
  - Can query database with *sst-info* utility



Source code for core:

- `sst-core/src/sst/core`

Source code for elements

- `sst-elements/src/sst/elements`
- Most elements have a tests/ directory
  - `sst-elements/src/sst/elements/<SomeComponent>/tests`
  - Often a good starting point for basic configurations

Basic SST Simulation

Simulation in just 3 steps… ☺

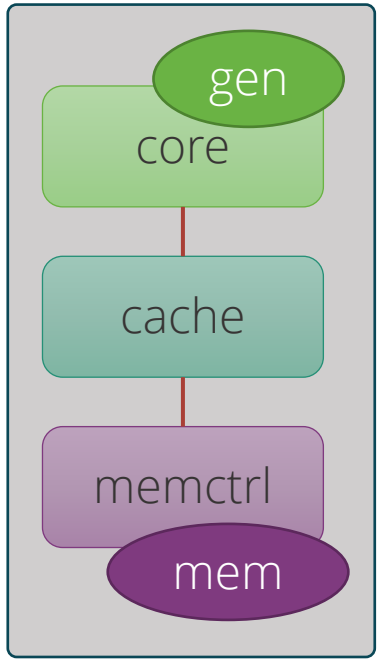
1. Define the system you want to simulate
2. Run SST
3. Analyze output

Input Files Describe the *Graph* of the System to Be Simulated

- The input file describes:
  - The **components** that make up the simulation (vertices) including their parameters
  - The **links** between the components and their latencies (edges)

- It also may:
  - Define any global options that control the simulation as a whole
  - Declare which statistics to collect and how to report them

- Typically written in Python
  - JSON also supported



Configuration File: Global SST parameters

Set any global simulation parameters

SST Python API
User-defined string
SST argument

Other options

- Most command line options to SST can also be set using `setProgramOption()`

```
import sst

##############################
## Define SST core options
##############################
# If this demo gets to 100ms, something
# has gone very wrong!
sst.setProgramOption("stop-at", "100ms")
```

Option	Definition
debug-file	File to print debug output to
heartbeat-period	If set, SST will print a heartbeat message at the specified period
print-timing-info	Tells SST to print timing information from the run
partitioner	Partitioner to use for parallel execution
output-partition	File to print partition to

Configuration File: Declare components

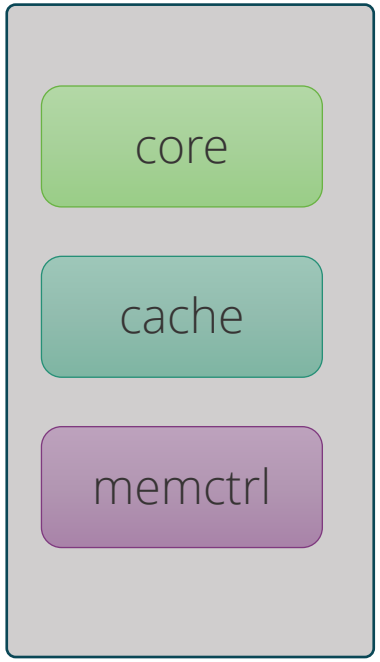
SST Python API
User-defined string
SST argument

Components: `sst.Component("name", "type")`

```
##########################
## Declare components
##########################
core = sst.Component("core", "miranda.BaseCPU")
cache = sst.Component("cache", "memHierarchy.Cache")
memctrl = sst.Component("memory", "memHierarchy.MemController")
```

Component name

Element library

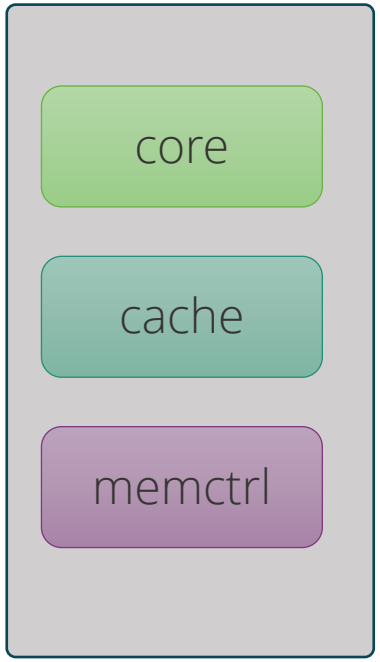


Configuration File: Configure the core

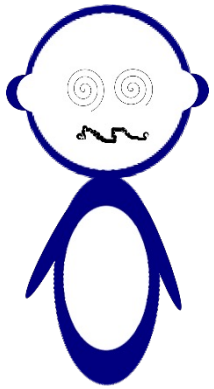
SST Python API
User-defined string
SST argument

Parameters: `addParams({ “parameter” : “value”, … })`

```
##########################################################
## Set component parameters and fill subcomponent slots
##########################################################
core.addParams({
    "clock" : "2.4GHz",
    "max_reqs_cycle" : 2,
})
```



How do I know what the options are?
Or even what elements I can pick from?



Aside: sst-info

Use sst-info to find information about all registered elements.

```
$ sst-info miranda.baseCPU

PROCESSED 1 .so (SST ELEMENT) FILES FOUND IN DIRECTORY(s)
[...]
================================================================================
ELEMENT LIBRARY 0 = miranda ()
Components (1 total)
   Component 0: BaseCPU
      Description: Creates a base Miranda CPU ready to execute an address
generator/access pattern
      ELI version: 0.9.0
      Compiled on: Dec  1 2023 14:33:14, using file: mirandaCPU.h
      Category: PROCESSOR COMPONENT
      Parameters (12 total)
         max_reqs_cycle: Maximum number of requests the CPU can issue per cycle
         (this is for all reads and writes)  [2]
         ...
```

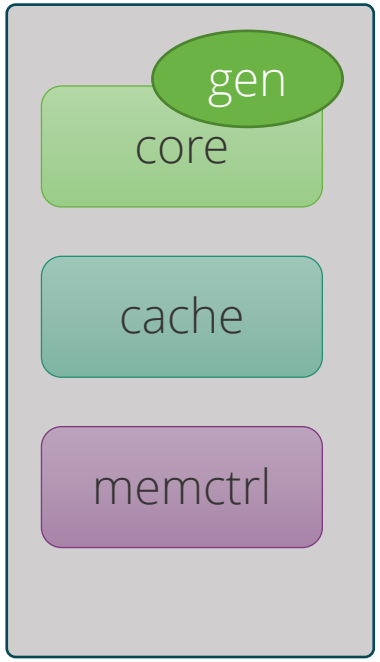
Configuration File: Add a subcomponent

SST Python API
User-defined string
SST argument

SubComponents: setSubComponent(“slotname”, “type”)

- Recall: SubComponent is a *swappable piece of functionality*

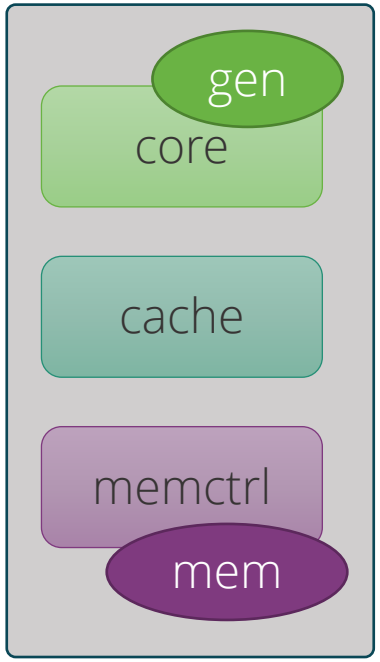
```
gen = core.setSubComponent("generator", "miranda.STREAMBenchGenerator")
gen.addParams({
    "n" : 1000, # Number of array elements
})
```



Configuration File: Configure the cache

SST Python API
User-defined string
SST argument

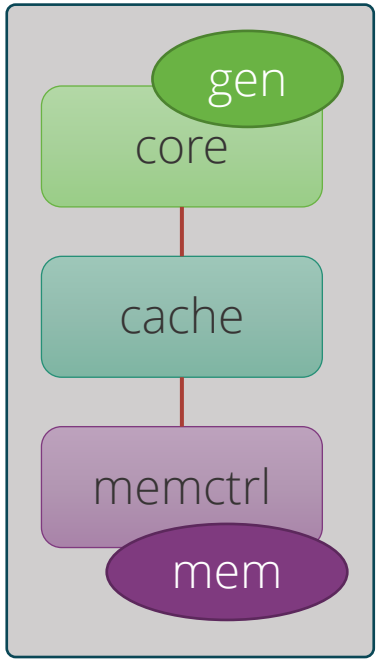
```
cache.addParams({
    "L1" : 1,
    "cache_frequency" : "2.4GHz",
    "access_latency_cycles" : 2,
    "cache_size" : "2KiB",
    "associativity" : 4,
    "replacement_policy" : "lru",
    "coherence_policy" : "MESI",
    "cache_line_size" : 64,
})
```



Configuration File: Configure the memory controller

SST Python API
User-defined string
SST argument

```
memctrl.addParams({
    "clock" : "1GHz",
    "backing" : "none", # No real memory values, just addresses
    "addr_range_end" : 1024*1024*1024-1,
})

memory = memctrl.setSubComponent("backend", "memHierarchy.simpleMem")
memory.addParams({
    "mem_size" : "1GiB",
    "access_time" : "50ns",
})
```



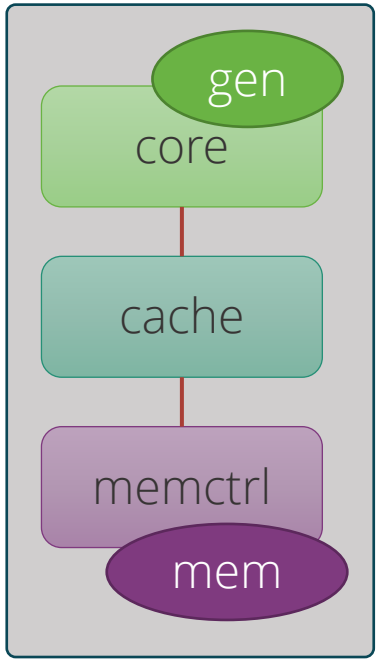
Configuration File: Declare and connect

SST Python API
User-defined string
SST argument

Links: `sst.Link("name")`

```
##########################################
## Declare links
##########################################
core_cache = sst.Link("core_to_cache")
cache_mem = sst.Link("cache_to_memory")

##########################################
## Connect components with the links
##########################################
core_cache.connect( (core, "cache_link", "100ps"),
                    (cache, "highlink", "100ps") )

cache_mem.connect( (cache, "lowlink", "100ps"),
                   (memctrl, ”highlink", "100ps") )
```

Link name



Configuration File: Connect links

SST Python API
User-defined string
SST argument

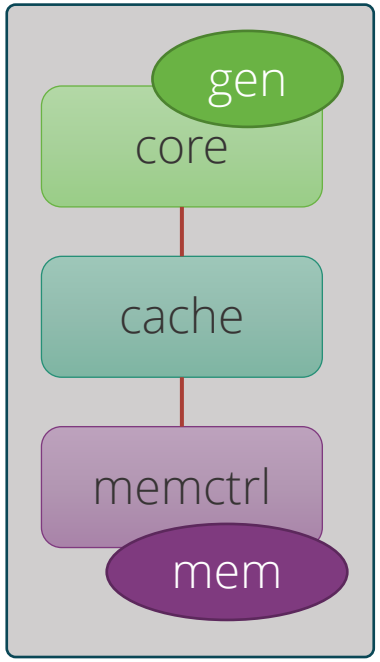
Connect components: `connect(endpoint1, endpoint2)`

- Where endpoint is: `(component, port, latency)`

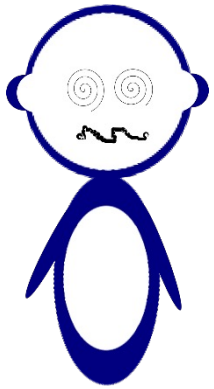
```
##########################################
## Connect components with the links
##########################################
core_cache.connect( (core, "cache_link", "100ps"),
                    (cache, ”highlink", "100ps") )

cache_mem.connect( (cache, "lowlink", "100ps"),
                   (memctrl, ”highlink", "100ps") )
```

Endpoint 1

Endpoint 2



How do I remember the port names?

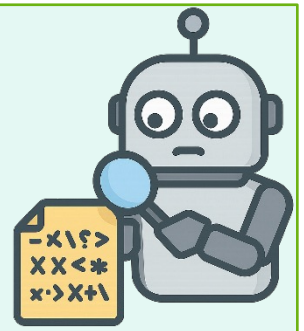


SSTInfo: Getting component info

sst-info: utility to query element libraries

```
$ sst-info memHierarchy.Cache
PROCESSED 1 .so (SST ELEMENT) FILES FOUND IN DIRECTORY(s) /home/sst/build/lib/sst
Filtering output on Element = “memHierarchy.Cache”
========================================================================
ELEMENT LIBRARY 0 = memHierarchy ()
Components (15 total)
   Component 0: Cache
      Description: Cache controller
…
      Parameters (34 total)
         cache_frequency: (string) Clock frequency or period with units (Hz or s; SI units OK) [<required>]
         cache_line_size: (uint) Size of a cache line (aka cache block) in bytes.  [64]
…
      Ports (12 total)
         highlink: Non-network upper/processor-side link (i.e., link towards the core/accelerator/etc.).
…
      Statistics (48 total)
         TotalEventsReceived: Total number of events received,  (units = "events") Enable level = 1
…
```

Optionally filter for a specific library and/or component



Parameter

Definition

“REQUIRED” or default value

Port name

Definition

Name

Definition

Units

Enable Level

Running SST

Usage: `sst [options] configFile.py`

Common options:

`-h | --help`	Print complete list of command line options
`-v | --verbose`	Print information about core runtime
`--debug-file <filename>`	Send debugging output to specified file (default: sst_output)
`--partitioner <self | simple | rrobin | linear | lib.partitioner_name>`	Specify the partitioning mechanism for parallel runs
`-n | --num_threads <num>`	Specify number of threads per rank
`--model-options "<args>"`	Command line arguments to send to the Python configuration file. Any arguments after a final – will be appended to the model-options.
`--output-partition <filename>`	Write partitioning information to <filename>

`--output-dot <filename>`
`--output-xml <filename>`
`--output-json <filename>`
	Output the configuration graph in various formats to <filename>

Running a simulation

Launch simulation

```
$ sst demo_1.py
```

Output

```
Simulation is complete, simulated time:  6.80711 us
```

We probably want more information about what happened though

- Can enable statistics in most components although details depend on implementation

```
$ sst-info memHierarchy.Cache
    …
Statistics (48 total)
        TotalEventsReceived: Total number of events received,  (units = "events") Enable level = 1
        CacheHits: Total number of cache hits, (units = count) Enable Level = 1
        latency_GetS_hit: Latency for read hits, (units = cycles) Enable level = 1
```

SST in parallel

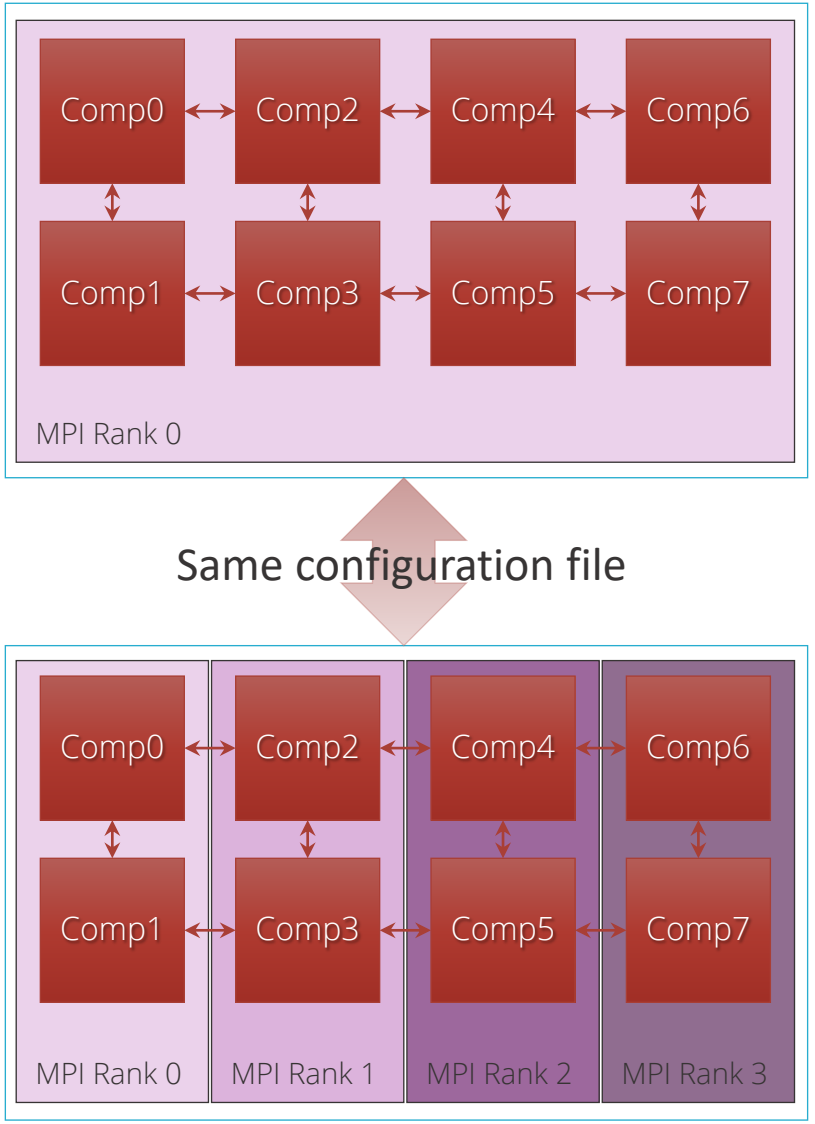
SST was designed from the ground up to enable scalable, parallel simulations

Components distributed among MPI ranks/threads

- Link latency controls synchronization rate

Sadly, MPI is not supported in our container

```
# Two ranks
$ mpirun -np 2 sst demo_1.py

# Two threads
$ sst -n 2 demo_1.py

# Two ranks with two threads each
# This will give a warning since we only
# have 3 components across 4 ranks/threads
$ mpirun -np 2 sst -n 2 demo_1.py
```



More Examples

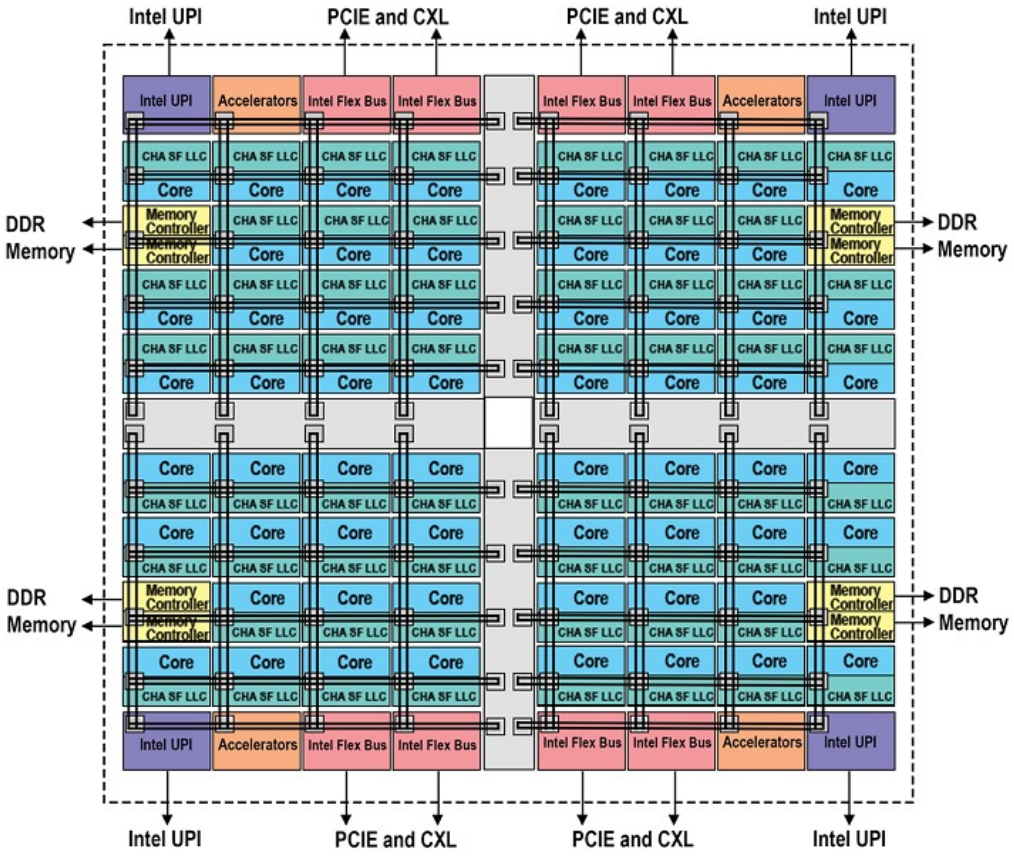
Tutorials are available at https://github.com/sstsimulator/sst-tutorials

Let’s skip a bit and look at something relevant to you…

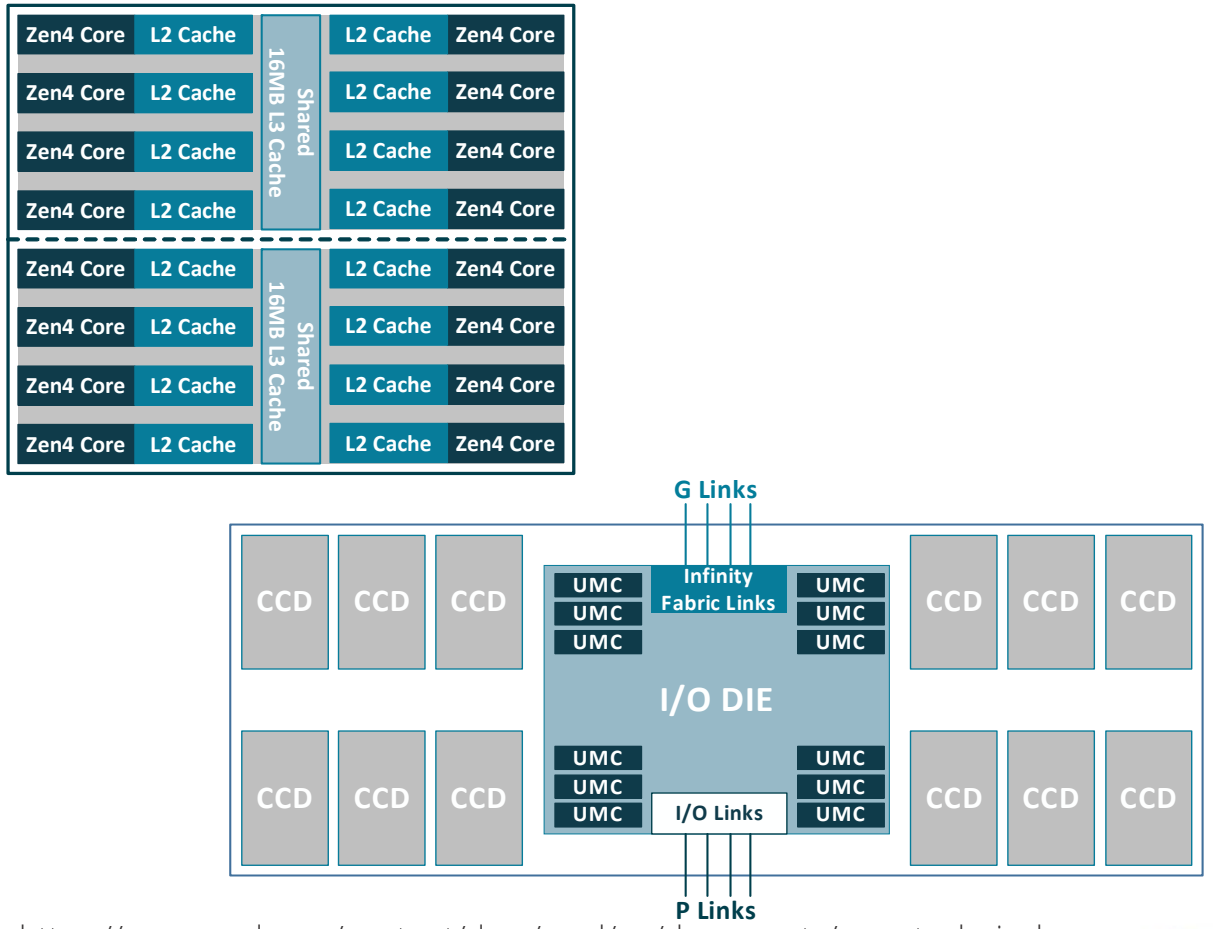
Node/SOC Simulations

Simulating SOCs

What if you wanted something much more complex? Can SST simulate a core with SMT? What about something like a modern processor? SPR? EPYC?



https://www.intel.com/content/www/us/en/developer/articles/technical/fourth-generation-xeon-scalable-family-overview.html



https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/58015-epyc-9004-tg-architecture-overview.pdf

Vanadis: OOO Processor

$ sst-info vanadis

- MIPS32 and RV64 compatible processor model
- OOO model
  - Configure micro-architectural details
    - Instruction fetch/decode/retire rate
    - ROB size
  - Branch prediction
- Multi-threaded cores
- Musl libc used to cross-compile programs
  - Also tested with clang and gcc
- System-call emulation

★ GPGPU-Sim Integration

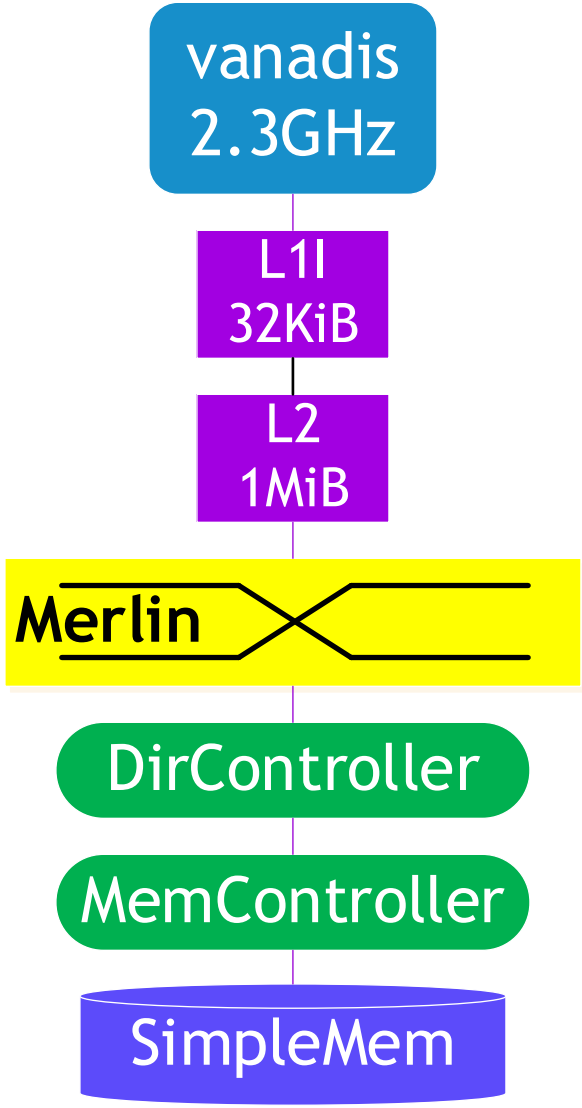
Single Core Example

Required elements

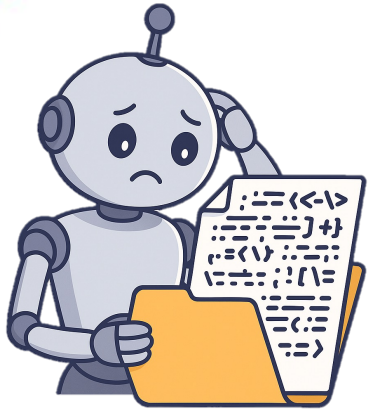
- Vanadis
- memHierarchy
- Merlin

Example located in:
../vanadis/tests/basic_vanadis.py

Test harness can instantiate multiple CPUs but this example assumes a single CPU



Global/System Parameters



```
# Tell SST what statistics handling we want
sst.setStatisticLoadLevel(4)
sst.setStatisticOutput("sst.statOutputConsole")

full_exe_name = os.getenv("VANADIS_EXE", "./small/" + testDir + "/" + exe +  "/" + isa + "/" + exe )
exe_name= full_exe_name.split("/")[-1]

verbosity = int(os.getenv("VANADIS_VERBOSE", 0))
os_verbosity = os.getenv("VANADIS_OS_VERBOSE", verbosity)
pipe_trace_file = os.getenv("VANADIS_PIPE_TRACE", "")
lsq_ld_entries = os.getenv("VANADIS_LSQ_LD_ENTRIES", 16)
lsq_st_entries = os.getenv("VANADIS_LSQ_ST_ENTRIES", 8)

rob_slots = os.getenv("VANADIS_ROB_SLOTS", 64)
retires_per_cycle = os.getenv("VANADIS_RETIRES_PER_CYCLE", 4)
issues_per_cycle = os.getenv("VANADIS_ISSUES_PER_CYCLE", 4)
decodes_per_cycle = os.getenv("VANADIS_DECODES_PER_CYCLE", 4)

integer_arith_cycles = int(os.getenv("VANADIS_INTEGER_ARITH_CYCLES", 2))
integer_arith_units = int(os.getenv("VANADIS_INTEGER_ARITH_UNITS", 2))
fp_arith_cycles = int(os.getenv("VANADIS_FP_ARITH_CYCLES", 8))
fp_arith_units = int(os.getenv("VANADIS_FP_ARITH_UNITS", 2))
branch_arith_cycles = int(os.getenv("VANADIS_BRANCH_ARITH_CYCLES", 2))

cpu_clock = os.getenv("VANADIS_CPU_CLOCK", "2.3GHz")

numCpus = int(os.getenv("VANADIS_NUM_CORES", 1))
numThreads = int(os.getenv("VANADIS_NUM_HW_THREADS", 1))

vanadis_cpu_type = "vanadis."
vanadis_cpu_type += os.getenv("VANADIS_CPU_ELEMENT_NAME","dbg_VanadisCPU")
```

Core Parameters

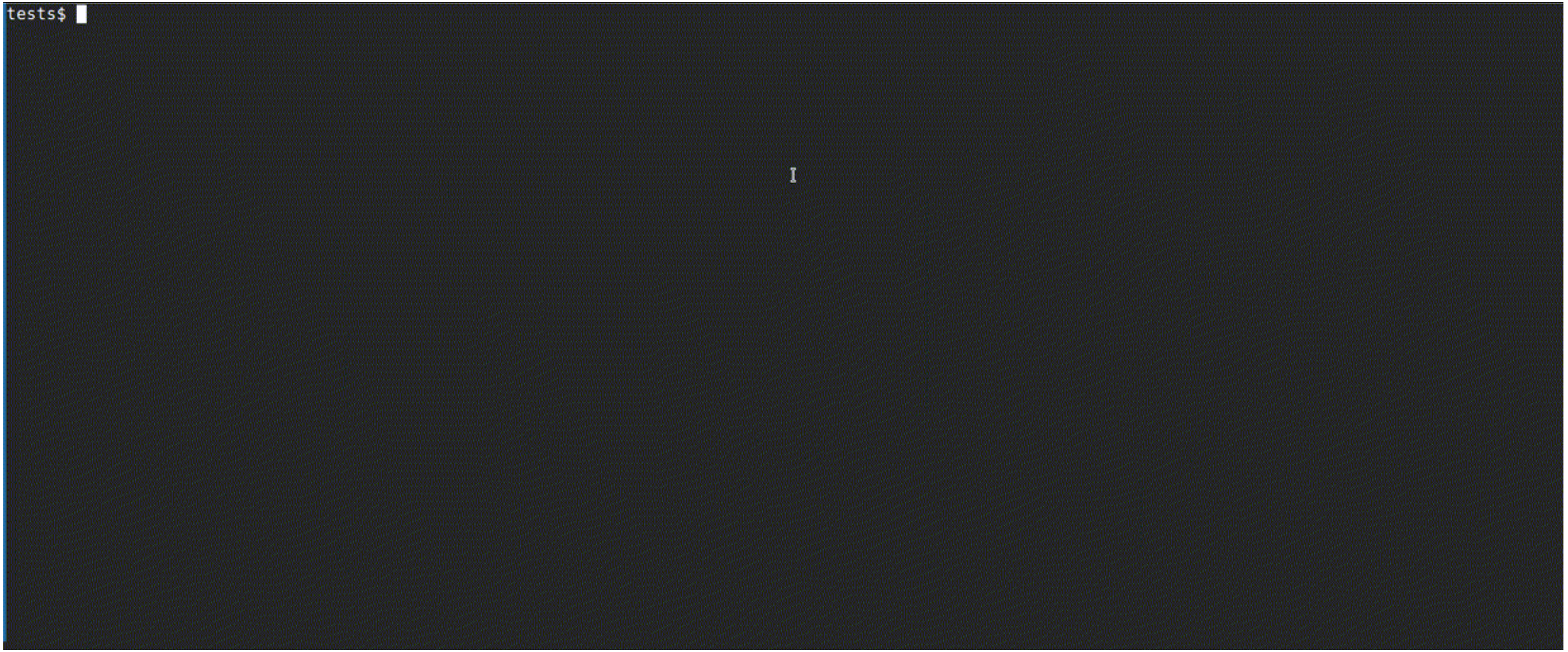
```
cpuParams = {
    "clock" : cpu_clock,
    "verbose" : verbosity,
    "hardware_threads": numThreads,
    "physical_fp_registers" : 168 * numThreads,
    "physical_integer_registers" : 180 * numThreads,
    "integer_arith_cycles" : integer_arith_cycles,
    "integer_arith_units" : integer_arith_units,
    "fp_arith_cycles" : fp_arith_cycles,
    "fp_arith_units" : fp_arith_units,
    "branch_unit_cycles" : branch_arith_cycles,
    "print_int_reg" : False,
    "print_fp_reg" : False,
    "pipeline_trace_file" : pipe_trace_file,
    "reorder_slots" : rob_slots,
    "decodes_per_cycle" : decodes_per_cycle,
    "issues_per_cycle" :  issues_per_cycle,
    "retires_per_cycle" : retires_per_cycle,
    "pause_when_retire_address" : os.getenv("VANADIS_HALT_AT_ADDRESS", 0),
    "start_verbose_when_issue_address": dbgAddr,
    "stop_verbose_when_retire_address": stopDbg,
    "print_rob" : False,
    "checkpointDir" : checkpointDir,
    "checkpoint" : checkpoint
}
```

Cache Parameters

```
l1dcacheParams = {
    "access_latency_cycles" : "2",
    "cache_frequency" : cpu_clock,
    "replacement_policy" : "lru",
    "coherence_protocol" : protocol,
    "associativity" : "8",
    "cache_line_size" : "64",
    "cache_size" : "32 KB",
    "L1" : "1",
    "debug" : mh_debug,
    "debug_level" : mh_debug_level,
}

l1icacheParams = {
    "access_latency_cycles" : "2",
    "cache_frequency" : cpu_clock,
    "replacement_policy" : "lru",
    "coherence_protocol" : protocol,
    "associativity" : "8",
    "cache_line_size" : "64",
    "cache_size" : "32 KB",
    "prefetcher" : "cassini.NextBlockPrefetcher",
    "prefetcher.reach" : 1,
    "L1" : "1",
    "debug" : mh_debug,
    "debug_level" : mh_debug_level,
}
```

```
l2cacheParams = {
    "access_latency_cycles" : "14",
    "cache_frequency" : cpu_clock,
    "replacement_policy" : "lru",
    "coherence_protocol" : protocol,
    "associativity" : "16",
    "cache_line_size" : "64",
    "cache_size" : "1MB",
    "mshr_latency_cycles": 3,
    "debug" : mh_debug,
    "debug_level" : mh_debug_level,
}
```

Example – *basic_vanadis.py*

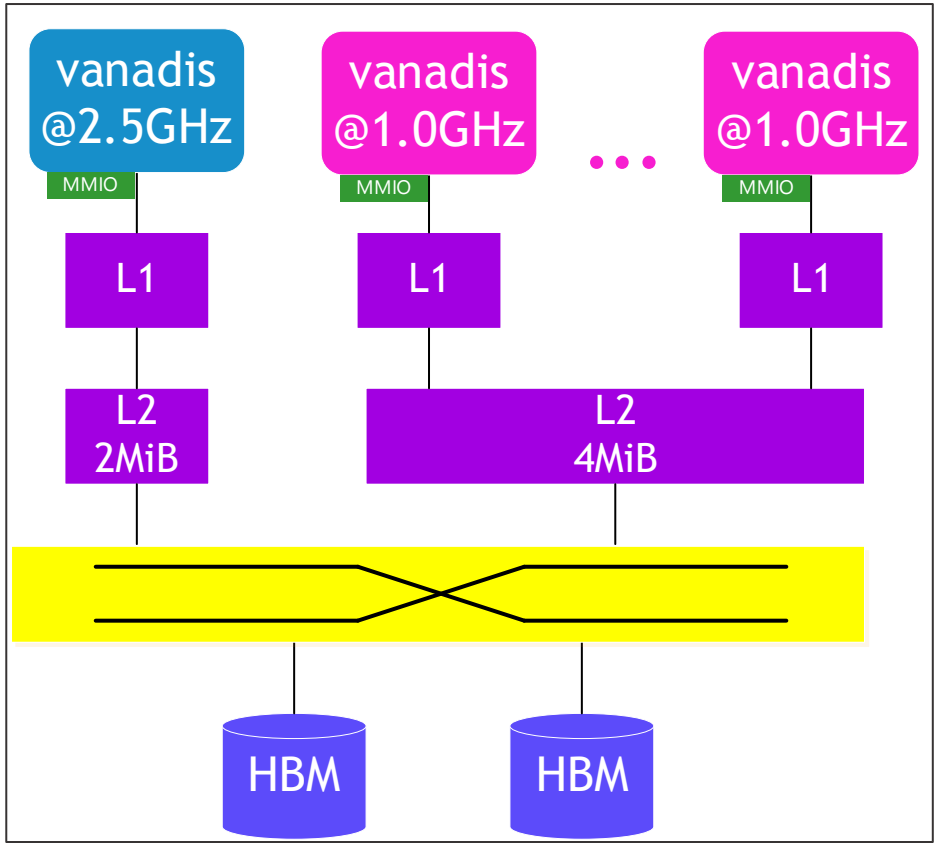


Other Configurations

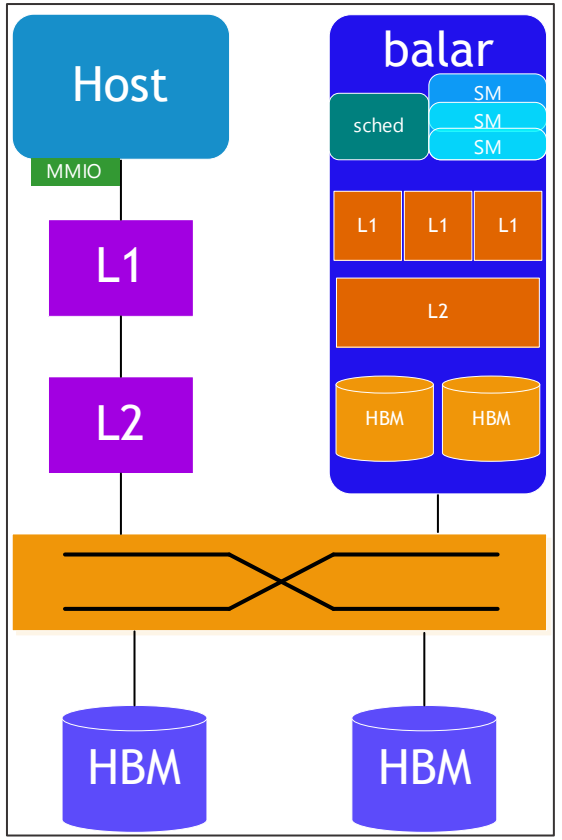
Flexibility and parameterization of vanadis allows it to be used in many different design scenarios

Integration with SST allows significant degree of flexibility

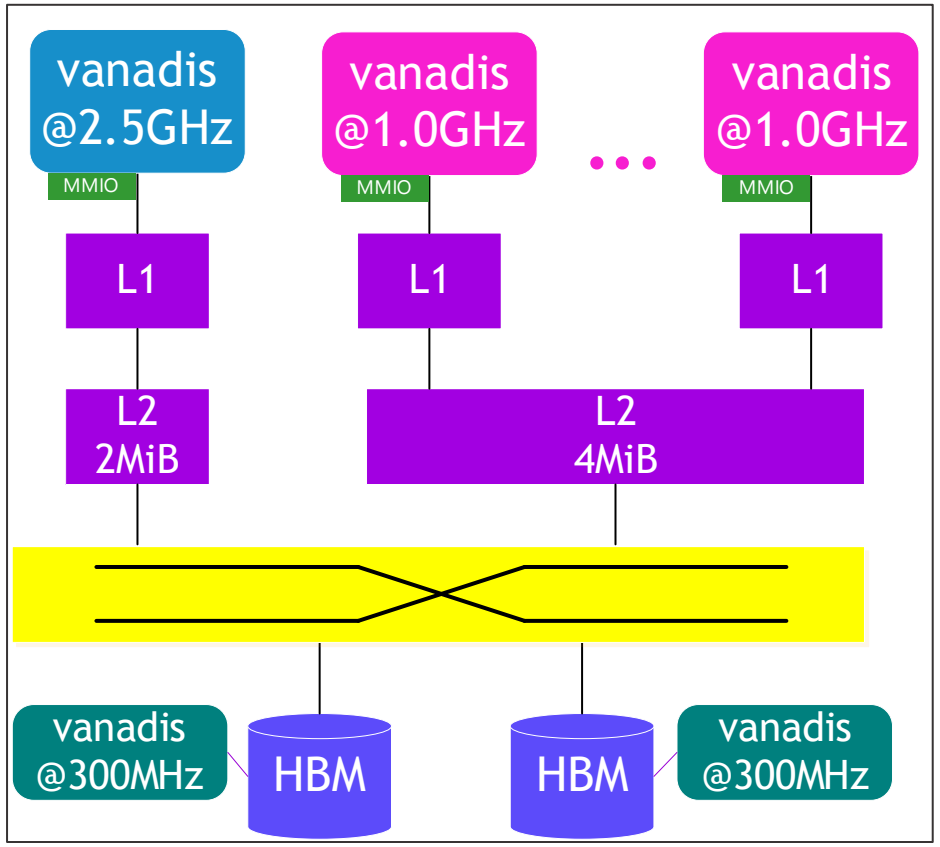
Potential for different cores to have different ISAs (*e.g.* PIM use case)



Mix-core Node



Accelerated Node



PIM Node

Accelerators and RoCC

Enable support for tightly-coupled accelerators

Leverages the Rocket Custom Coproeccoer (RoCC) interface

- Takes advantage of the custom instruction encoding space within the RISC-V ISA while standardizing the control signals used by the accelerator
- Enables early-stage software development targeting the accelerator
  - An essential capability since software support is often a major blocker in the deployment of systems with custom accelerators

Currently being tested with integration of analog Matrix Vector Multiplication (MVM) components (golem)

- Modified LLVM compiler, initial software stack and application development has begun
- https://github.com/sstsimulator/sst-elements/tree/master/src/sst/elements/golem

Getting Help & Extending SST

Extending SST

SST was designed for extensibility

- Components/subcomponents can be added without touching SST Elements
  - Example: write a new prefetcher and have memH caches use it → *no changes* to memHierarchy
- SST-Core APIs are stable → one year deprecation period
  - Element APIs may be less so but generally try to keep them consistent
- Many users start with SST Elements and then build their own customized libraries
  - Partially or completely replacing SST Element functionality

Many approaches to using SST

- Core only: Write your own components from scratch
- Start from existing Elements and replace components/subcomponents to meet your needs
- Wrap existing simulators and insert as components or subcomponents

Extending SST: Resources

Example element library

- Components demonstrating links, ports, clocks, event handling, etc.
- sst-elements/src/sst/elements/simpleElementExample/

simpleSimulation

- Simulates a car wash (a little more complex than example elements)

Example external element library

- Demonstrates building and registering a new element library
- https://github.com/sstsimulator/sst-external-element

Finally: Getting help

SST website contains lots of information (www.sst-simulator.org)

- Downloading, installing, and running SST
- Element libraries and external components
- Guides for extending SST
- Information on APIs
- Information about current development efforts
- Past tutorial slides and exercises

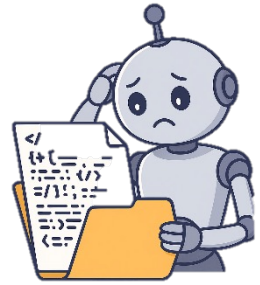
SST Github

- Current development
- Issues track user questions as well as development plans, bugs, etc.


QUESTIONS

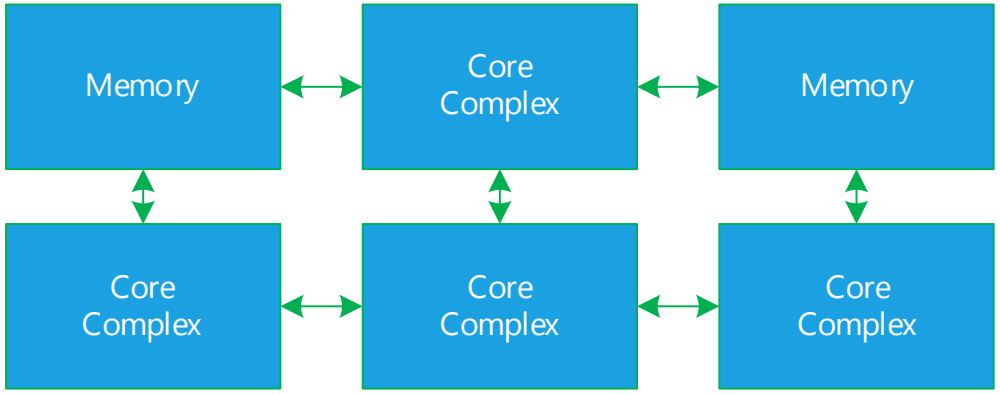
Backup

Multi-core Vanadis



What if we want to make something more complex?

- 3x2 mesh
- CPUs at 1, 3, 4, 5
- Memory at 0, 2




```
$ cat boom_vanadis-kingsley.py
```

```
build_mesh()
for y in range(0, mesh_stops_y):
        for x in range (0, mesh_stops_x):
         …
        kRtrData.append(sst.Component("krtr_data_" + str(nodeNum), "kingsley.noc_mesh"))
for y in range(0, mesh_stops_y):
        for x in range (0, mesh_stops_x):
             …
            ## North-south connections
            ## East-West connections
node_builder.build_endpoint(i, kRtrReq[i], kRtrAck[i], kRtrFwd[i], kRtrData[i])
```

Merlin: Network Simulator

Low-level, flexible networking components that can be used to simulate high-speed networks (machine level) or on-chip networks

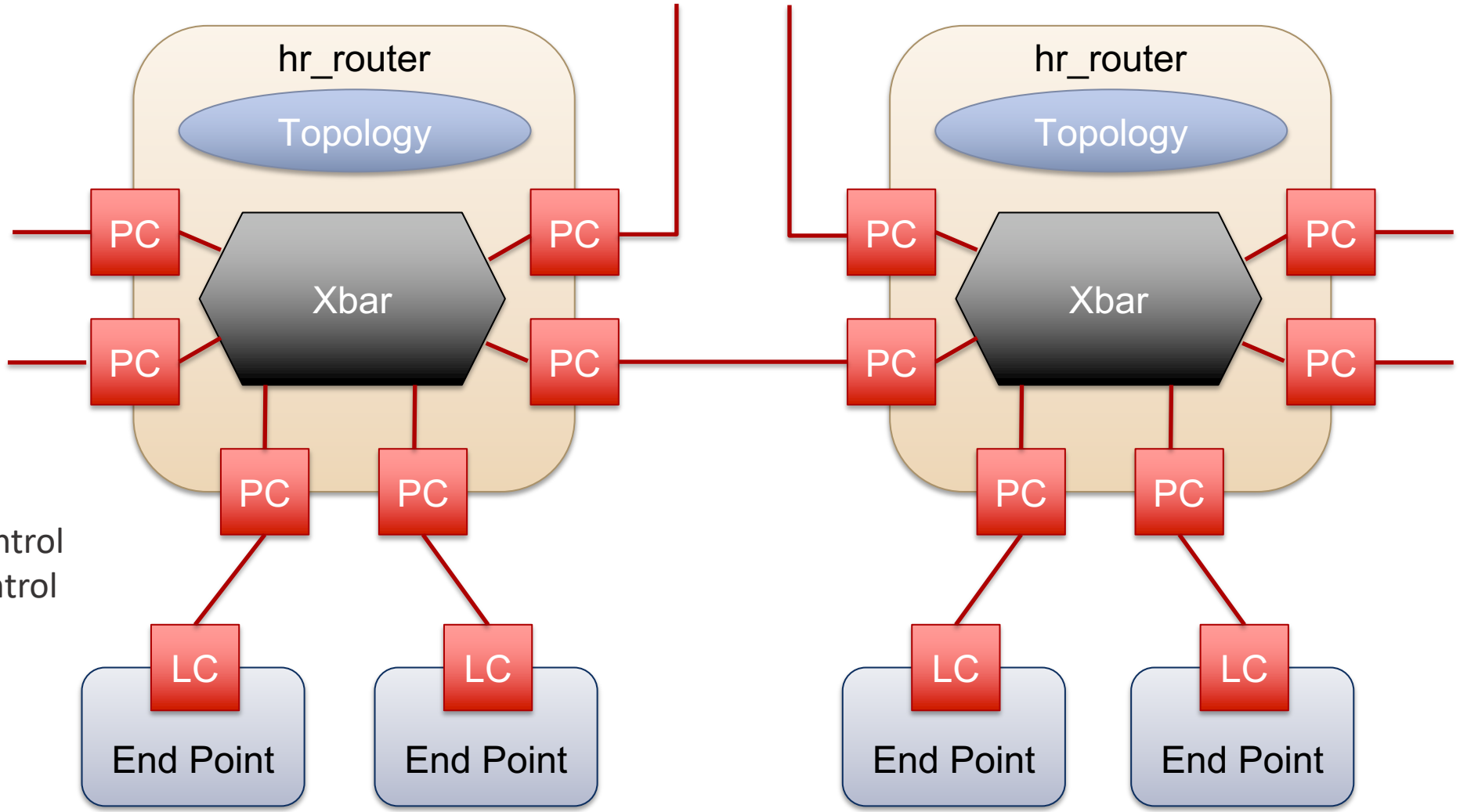
Capabilities

- High radix router model (hr_router)
- Topologies – mesh, n-dim tori, fat-tree, dragonfly, hyperx

Many ways to drive a network

- Simple traffic generation models
- *MemHierarchy*
- Lightweight network endpoint models (*Ember*)
- Application skeletons (*Mercury*)
- Or, make your own

Merlin High Level Overview



PC = PortControl
LC = LinkControl

General Parameters

General

- link_bw – bandwidth of the network links (in B/s or b/s)

hr_router

- xbar_bw – per port bandwidth into router crossbar (in B/s or b/s)
- flit_size – size of a flit (in B or b)
- input_latency – input latency of router (in s)
- output_latency – output latency of router (in s)
- xbar_arb – crossbar arbitration unit to be used

hr_router/LinkControl

- input_buf_size – size of input buffer for router/NIC (in B or b)
- output_buf_size – size of output buffer for router/NIC (in B or b)

LinkControl-PortControl Interactions

LinkControl and PortControl share data and negotiate various parameters during the init() phase

- Each LC/PC pair will negotiate link bandwidth. It is set to the minimum of the two set bandwidths
- PortControl will notify the LinkControl of the network ID used to address it
- PortControl will report FLIT size to the LinkControl
- LinkControl notifies PortControl of the desired Virtual Networks to be used
  - This is a deprecated feature. Number of VNs is now set directly through the router
- The LinkControl and PortControl objects manage send credits

ReorderLinkControl

Inherits from SST::Interfaces::SimpleNetwork

Contains a SimpleNetwork interface object to talk to the “physical” layer (this is typically just a LinkControl).

Puts a sequence number on each packet and reorders packets on the receive-side before giving them to endpoint

- Allows endpoint models that can’t handle out of order receipt of packets to use network models that don’t guarantee ordering

Currently assumes “infinite” resources for buffer and reordering

Crossbar Arbitration

xbar_arb_rr

- Round robin allocation – first across ports, then across virtual channels

xbar_arb_lru

- Least recently used – across all port/VC pairs

xbar_arb_age

- Oldest packets get higher priority

xbar_arb_rand

- Random priority assigned to each port/VC pair each arbitration cycle

Topology Module

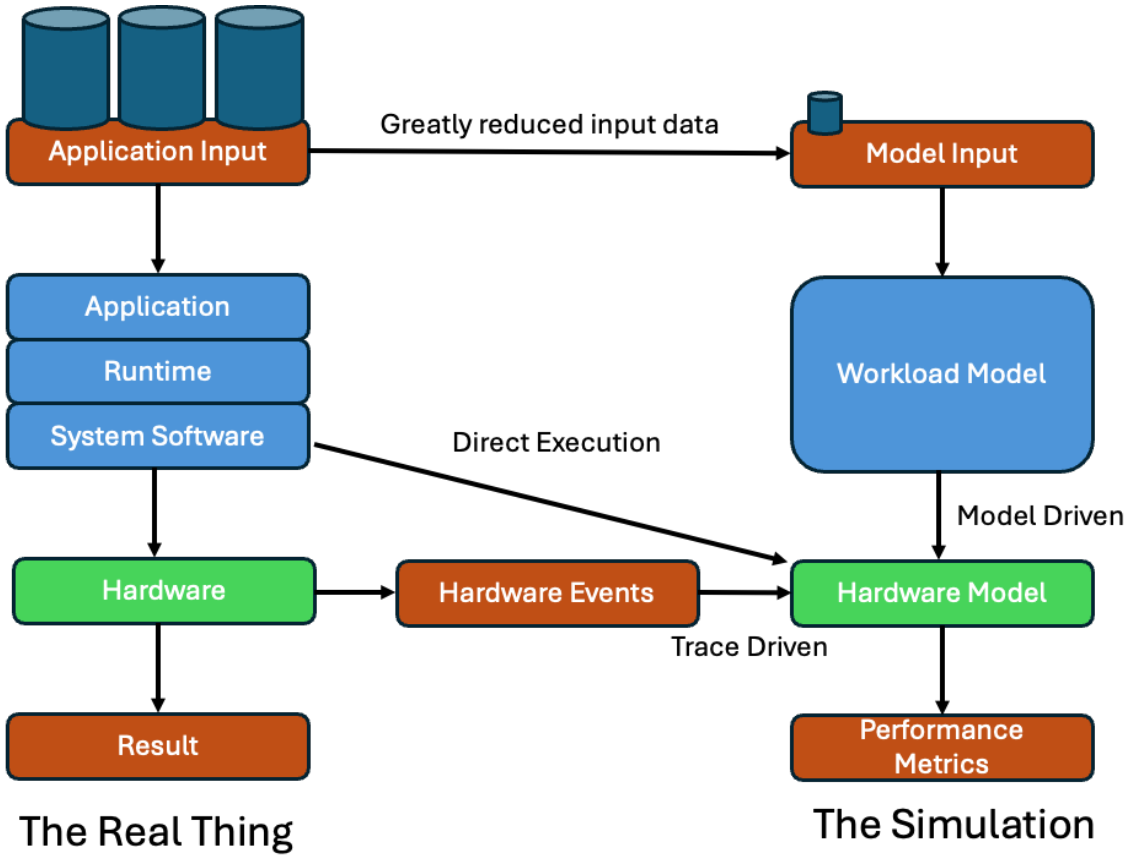
Router knows nothing about topology/routing without loading a topology module

- Can use static and/or dynamic routing

Available topologies

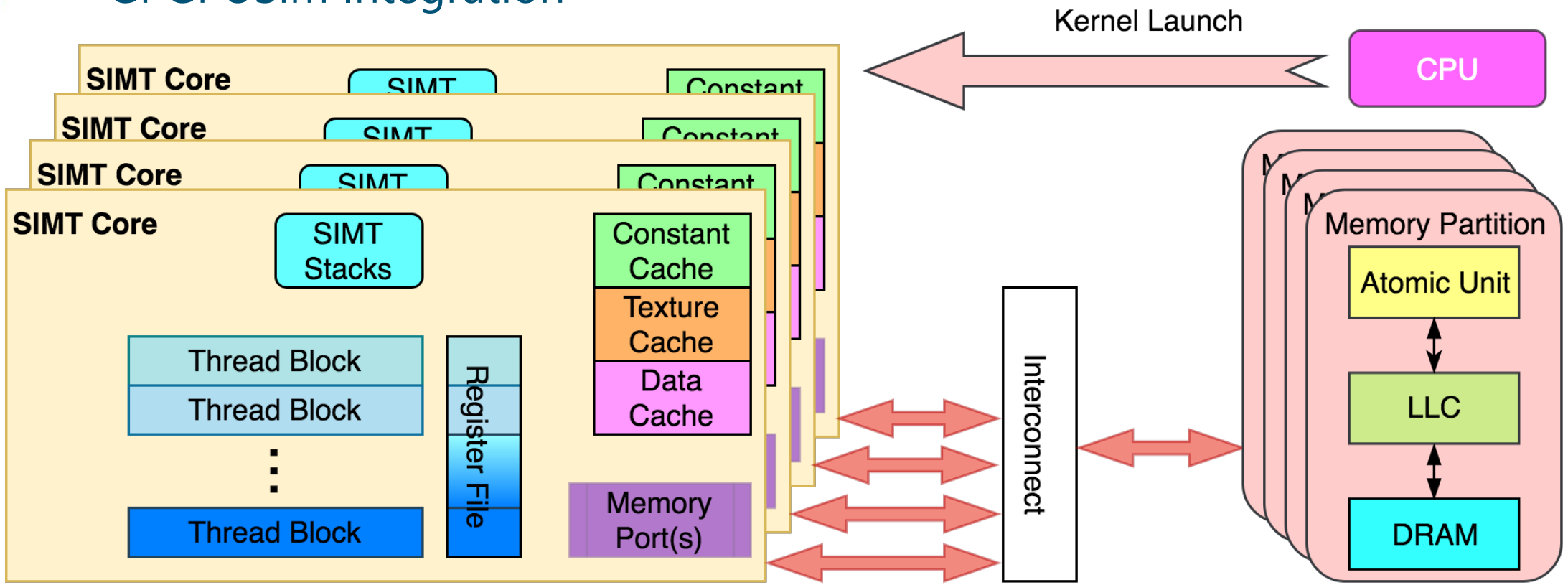
- SingleRouter
- Mesh
- Torus
- Fattree
- HyperX/Flattened Butterfly
- Dragonfly
- PolarFly/PolarStar

Simulation Approaches



- Direct execution is unachievable for full system scale
- Traces require full execution at target scale or some way to scale up traces accurately (difficult)
- Ideal workload models use parameterized representation to reproduce data movement and computation – **full results not computed**

GPGPUSim Integration



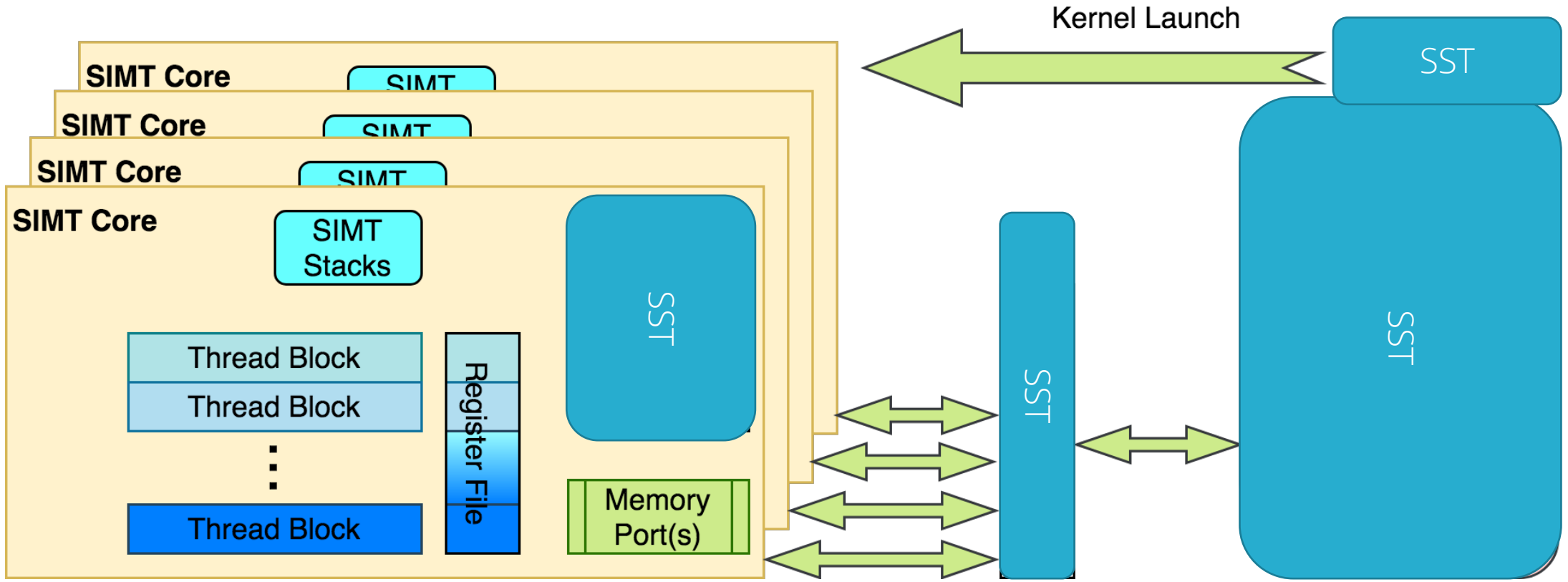
Functional

- SIMT units track default functional model from GPGPUSim

Timing

- SIMT units maintain execution timing; Booksim is used for the interconnect; and simple timing models are used for the memory partition

GPGPUSim Integration



Plan to keep the SIMT units and replace everything else

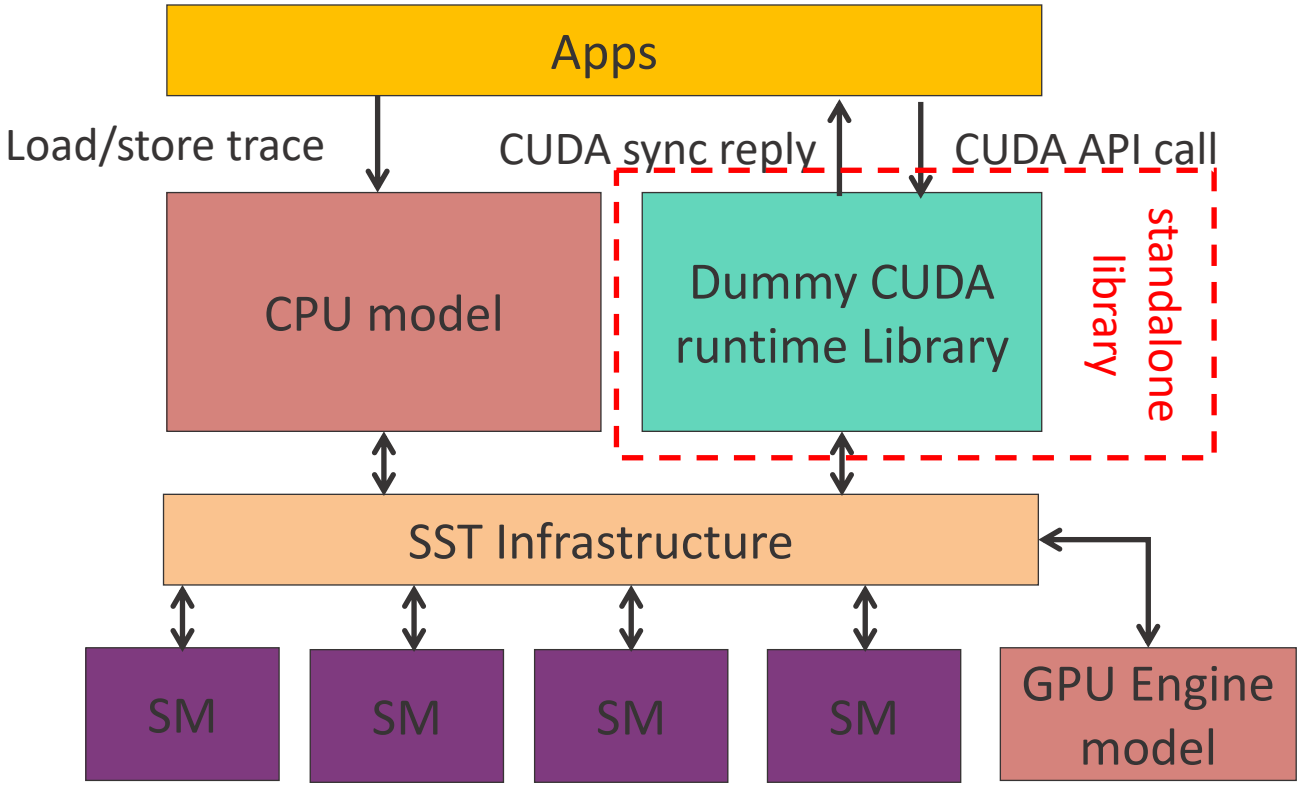
- CPU → SST execution component (Vanadis, Ariel, etc.)
- Interconnect → SST networking component (Shogun, Merlin, Kingsley, etc.)
- Memory Partition & Caches→ SST memHierarchy component for caches and various other backends for the backing store (DRAMSim, SimpleMem, Cramsim, etc.)

CUDA API interception model

Standalone dummy CUDA lib which is independent of CPU model

Applications link dynamically with lib (shared object)

Dummy lib interacts with GPU engine for kernel launch and CUDA memory copies

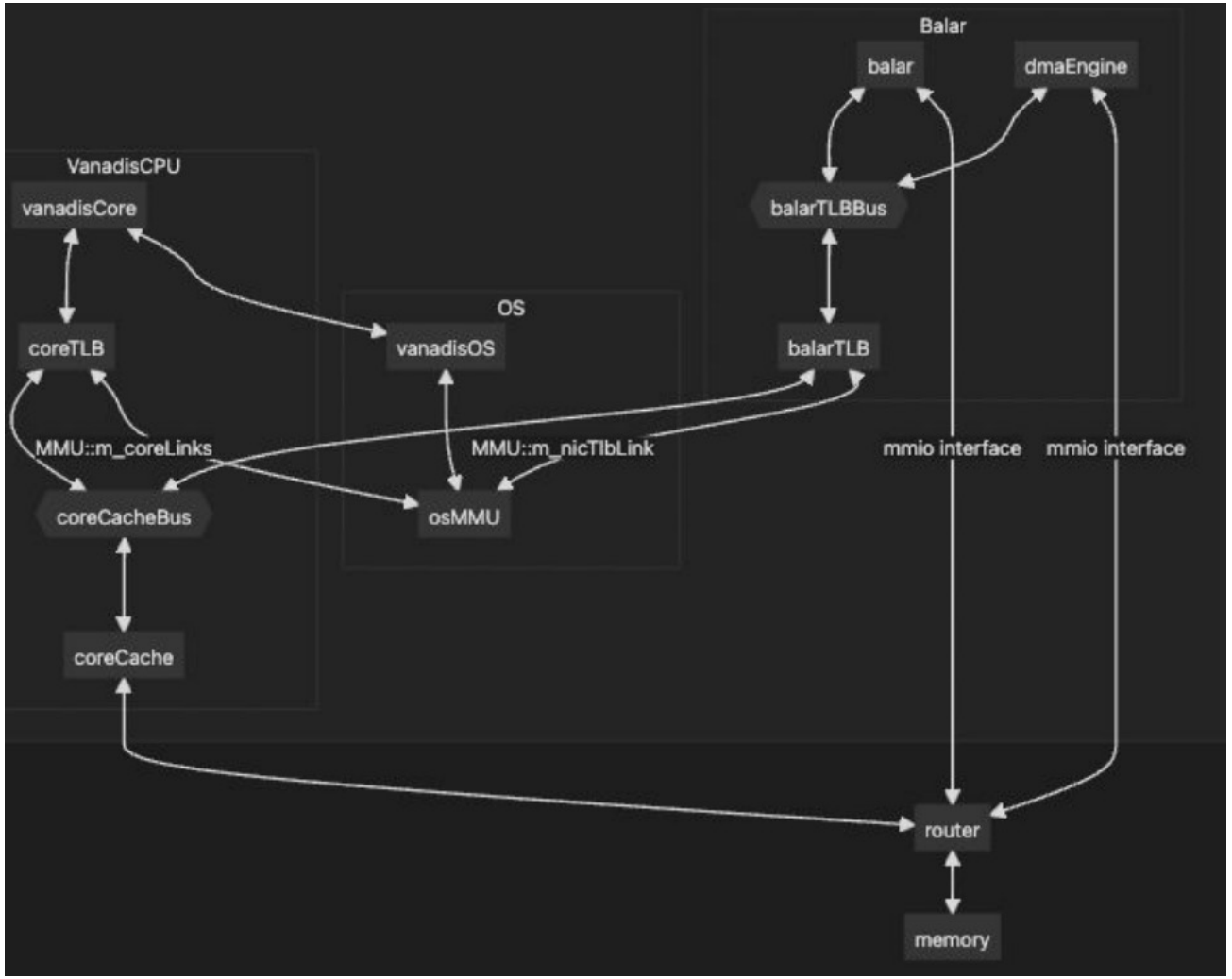


API Calls Forwarded to GPU Model	
__cudaRegisterFatBinary	cudaFree
__cudaRegisterFunction	cudaLaunch
cudaMalloc	cudaGetLastError
cudaMemcpy	cudaRegisterVar
cudaConfigureCall	cudaSetupArgument
cudaOccupancyMaxActiveBlocksPerMultiprocessorWithFlags	

balar Integrated With *vanadis* via MMAP

When coupled with vanadis, RVA23+CUDA binaries can be run

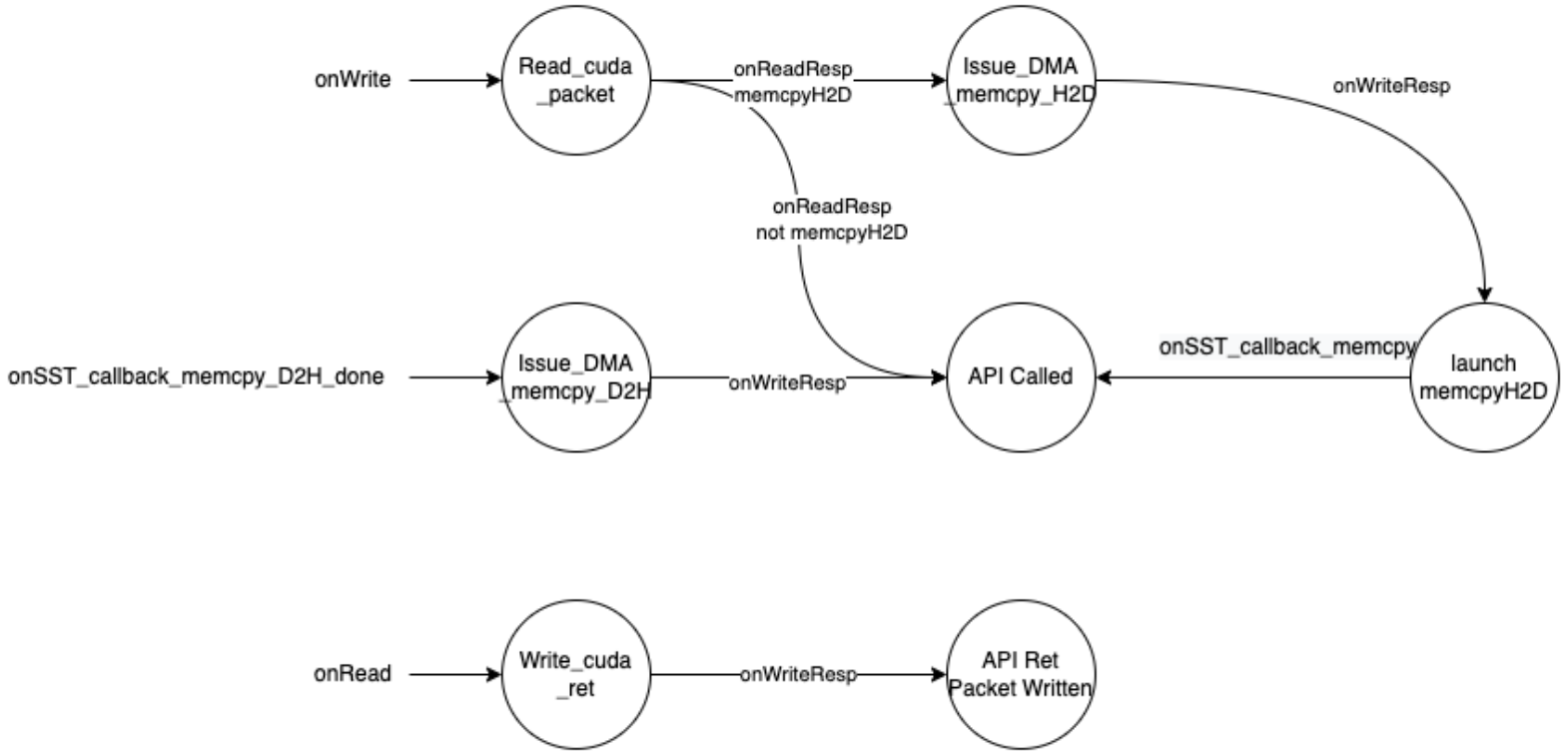
- Add balar as a device in VanadisOS with file descriptor being -2000 at 0x8010000
- balar and the dmaEngine have both mem_iface (for data packets) and mmio_iface (for command packets)



BalarMMIO

Responsible for relaying CUDA api requests from SST to GPGPU-Sim.

Currently it supports running with CUDA traces without a real CPU model (with BalarTestCPU) or with Vanadis core



Slide Left Blank